Quantum Sensing Agent: An LLM Agent that Operated a Scanning-Confocal NV-Center Experiment in Early 2025

Zechen Zhang* Ruolan Xue Jack Fan Amir Yacoby
Department of Physics, Harvard University, Cambridge, MA 02138, USA

Abstract

Large-language-model (LLM) agents are increasingly used to run laboratory instruments. We report the Quantum Sensing Agent (QSA), an LLM agent that operated a room-temperature scanning-confocal setup for nitrogen-vacancy (NV) centers in diamond starting in February 2025; to our knowledge, this makes it among the earliest LLM agents to operate a quantum-sensing experiment. QSA chains confocal scans, zoomed emitter searches, focusing, and continuous-wave optically detected magnetic resonance (ODMR). It changes only parameter values in JSON configurations, runs four allow-listed measurement scripts, and reads the resulting plots with a vision model, under the supervision of an experimenter. Later versions added explicit assistant and auto modes and retrieval over past sessions and their plots, intended to let the agent learn from earlier attempts. Over 16 hardware sessions on 10 days (18 February – 16 June 2025) the agent completed 73 measurements. In the best session it worked through six operator-selected candidates over 3.4 hours, proposed causes when two showed no resonance, and recorded an ODMR spectrum with two dips near 2.863 and 2.878 GHz on the third; across the deployment it turned all 22 numerical instructions from the operator into exactly the requested configurations. We also document its failures: low ODMR contrast, unreliable reading of coordinates from images, no observed benefit from retrieval on the hardware, and LLM replies that invented tool output, which in a short unsupervised test led the agent to report fabricated results as its own progress. We distill these observations into lessons for building agents for quantum laboratories.

1 Introduction

Running a quantum-sensing experiment involves a long chain of routine but judgment-laden steps: map a sample, pick a candidate emitter, centre on it, focus, choose a microwave window and power, decide whether a flat spectrum means a bad emitter or a bad setup, and move on. Classical automation optimizes *within* one of these steps, for example by choosing ODMR frequencies adaptively [9] or focusing on point emitters [22]. Deciding *which* step to run next, and interpreting a failure in words, has remained the experimenter's job. LLM agents that call tools are a natural candidate for this role. They had already planned and executed chemistry experiments [2, 4], operated scanning-probe microscopes [16, 18], and calibrated a superconducting quantum processor [5].

This report documents QSA, an LLM agent (Claude, via the Anthropic API) that operated a scanning-confocal NV-center ODMR setup in the Yacoby lab at Harvard. QSA ran its first hardware session on **18 February 2025**, its main deployment ended on 14 May 2025, and a last, short auto-mode test ran on 16 June 2025. We describe the system as built and the deployment as logged.

To our knowledge, QSA is among the earliest demonstrations of a large-language-model agent operating a quantum-sensing experiment. Its first session on an NV-center confocal setup took place on 18 February 2025; this date rests on the timestamps inside the session log `agent_history.log`, which entered the repository in its first commit on 3 April 2025 (`17c17e1`). The earliest NV-center LLM-agent study we found was posted in July 2026 [13]. The earliest LLM-agent quantum-sensor study on any platform we found was posted in August 2025 [25]. Comparable later systems go further than QSA. Isogawa et al. [13] run pulsed Ramsey and CPMG measurements autonomously over many hours. QSA ran CW ODMR with a human at the setup

*Correspondence: zechen_zhang@g.harvard.edu

directing each step; per-action approval was enforced in the February–April versions. We also note that this comparison is asymmetric: our date comes from internal logs, while other groups' dates are arXiv posting dates, and their experiments may have run earlier.

Contributions.

1. **A closed-loop agent on a real NV confocal-ODMR setup.** QSA chains a galvo scan, a zoomed emitter search, a z-focus optimization and a CW-ODMR sweep, writing each configuration and reading each plot itself, with the operator choosing candidates and frequency windows (Sections 3 and 4.2).
2. **A conservative tool design.** The agent can only edit parameter values in JSON configurations, run four allow-listed scripts in a fixed command format, and read or write inside two folders. In the February–April versions every write, run and vision action needed the operator's approval (Section 3.3).
3. **Memory of past sessions and their plots.** Sessions are embedded on exit together with the names of the plots they refer to, so a later session can retrieve earlier attempts and reopen the old figures with vision (Section 3.4). We present this as a design feature: it was built and tested in software, and no hardware log shows it changing behaviour.
4. **A deployment record with lessons learned.** 73 completed measurements in 16 hardware sessions on 10 days (18 February – 16 June 2025), with a 14 May case study, statistics on guidance, permissions and failures, and an analysis of what went wrong, including an unsupervised run that believed its own invented results (Section 4).

2 Related work

LLM agents operating laboratory instruments. Coscientist [2] and ChemCrow [4] established the "LLM plus tools plus instrument API" pattern in chemistry; self-driving laboratories more broadly are reviewed in [28]. In microscopy, Liu et al. [16] used GPT-4 with instrument APIs for scanning-probe workflows, and AILA [18] ran atomic force microscopy with a multi-agent LLM and found frontier models fragile and prone to deviating from instructions. That finding motivates QSA's restrictive tool layer. Later agents at beamlines, accelerators and cryogenic transport setups are listed in Appendix C.

LLM agents for quantum hardware. k-agents [5], posted in December 2024, calibrated a superconducting processor before QSA; it is the reason we do not claim to be the first LLM agent on quantum hardware. Later work on other quantum platforms is summarised in Appendix C. For quantum sensing specifically we found two studies. QCopilot [25] optimizes laser cooling for a cold-atom sensor and keeps a knowledge base of past experiments. Isogawa et al. [13] operate a confocal NV setup with persistent project records and run Ramsey and CPMG sequences; theirs is the closest work to ours. Classical automation of NV experiments, such as adaptive Bayesian ODMR [9], optimizes inside a fixed protocol (Appendix C); QSA instead decides which protocol to run next, and the two are complementary.

Agent memory. Retrieval-augmented generation [14] and episodic or reflective memories [21, 27, 32, 35] let agents reuse past experience; simple trajectory retrieval is a strong foundation for agent memory, especially when combined with fine-tuning [11]. QCopilot, Vriza et al. and Isogawa et al. keep lab memories, and all three appeared after QSA's February 2025 deployment. QSA additionally stores, with each retrieved chunk, the names of the plots it refers to, so that the agent can reopen those figures.

3 System

3.1 Apparatus and physics

The negatively charged NV center has a spin-1 ground state whose $m_s = \pm 1$ levels lie $D \approx 2.87\,\mathrm{GHz}$ above $m_s = 0$ at room temperature. A field component $B_\parallel$ along the NV axis splits the ± 1 levels by $2\gamma B_\parallel$, with $\gamma \approx 2.8\,\mathrm{MHz/G}$. Green excitation pumps the spin into $m_s = 0$, and the ± 1 states fluoresce less. A resonant

microwave field therefore produces a dip in the count rate, and sweeping the frequency gives the ODMR spectrum [8]. All spin measurements in this project are such CW sweeps; the repository contains no pulsed sequences.

The setup is a room-temperature scanning-confocal microscope controlled by the lab's Python stack (`pylabcontrol` with `b26_toolkit`) on a Windows PC. The agent drives four thin command-line wrappers, each of which loads a JSON configuration, runs the corresponding toolkit class, and saves a PNG plot and a JSON data file: `galvo_scan` (a raster of the field of view), `find_nv` (a zoomed scan with a 2D Gaussian fit around a chosen point), `optimize` (z-focus) and `ESR` (a CW microwave sweep); hardware and defaults are in Appendix A. The intended workflow, which the system prompt states, is galvo scan → choose a spot → `find_nv` → `optimize` → ESR, repeated for each NV.

3.2 Agent loop and tools

Figure 3 (Appendix) shows the loop. The prompt consists of a system instruction (abridged in Appendix D) followed by the session transcript. The model must answer with one `<think>` block, containing its reasoning, and `<action>` blocks of JSON whose `type` is `message` (talk to the operator), `read` or `write` (read a configuration, or write a modified one into the run's `configs/` folder), `run` (launch one of the four scripts), `vision` (send a saved plot to the vision model) or, from June 2025, `rag_search` (retrieve past sessions). Script standard output and vision results are appended to the transcript, so the next LLM call sees them. Every session gets a run folder with `configs/`, `data/` and `logs/`. The log records each operator message, raw LLM reply, extracted reasoning, action, permission decision and script output with a timestamp, so every measurement can be traced to the configuration that produced it. The deployment analysis in Section 4 rests entirely on these logs.

Models. Every version calls Claude through the Anthropic API: Claude 3.5 Sonnet in February 2025, and Claude 3.7 Sonnet, including image analysis, from March 2025 onward (authors' record). The logs do not record the model. The committed code (April 2025 on) defaults to Claude 3.5 Sonnet for text and Claude 3.7 Sonnet for vision, so the text model of a later session depended on a local setting that we cannot check; we keep the authors' record as the default.

Prompting and vision. The system instruction (Appendix D) tells the agent to read the default configuration first and change parameter values only. Until June the history was serialized into a single flat `User:/Assistant:` transcript; Section 4.4 shows why this mattered. Plots of every type are read through the `vision` action.

3.3 Safety and permission model

The agent never touches instrument drivers directly. Four restrictions define what it can do:

1. **Configuration-only edits.** The agent changes measurement settings only by writing JSON files. The prompt requires it to keep the exact keys, structure and types of the default configuration.
2. **Allow-listed scripts.** A `run` command is parsed against a fixed pattern, `py <name>.py -config <file> -output-dir <dir>`, and the script is checked against an allow-list of the four wrappers; anything else is rejected with an error message that is fed back to the agent.
3. **Folder whitelist.** Reads and writes are restricted to the `configs` and `data` folders.
4. **Permission gate.** In the February–April versions, every write, run and vision action waited for an explicit "yes" from the operator at the console, and refusals were fed back to the agent. In the logs, the whitelist and allow-list rejected 9 reads, 3 writes and 4 run commands, all in short development sessions.

The version used on 14 May logged no permission prompts. There, write and run actions executed in the same turn in which the agent asked "May I proceed?". Control was turn by turn instead: every LLM call followed an operator message. The June 2025 code makes the gate explicit as an *assistant* mode and adds an *auto* mode that grants permissions, logs them, and stops after at most 10 consecutive steps. June versions

Table 1: Completed hardware measurements by date and type. Only "saved data" lines inside logged script output are counted. Two further galvo scans (12 May and 14 May 07:58) are recorded only in embedding snapshots, whose run logs are not in the repository, and are not included.

Date (2025)	galvo scan	`find_nv`	optimize (z)	CW ESR	total
18 Feb	1	1	0	0	2
19 Feb	1	4	1	3	9
28 Feb	4	3	0	0	7
1 Mar	0	5	1	2	8
6 Mar	1	0	0	0	1
11 Mar	4	0	0	0	4
12 Mar	5	0	1	1	7
19 Apr	1	4	1	4	10
14 May	2	10	4	8	24
16 Jun	1	0	0	0	1
Total (10 days, 16 sessions)	**20**	**27**	**8**	**18**	**73**

also execute only the first action in each reply. The only hardware session with the June code, on 16 June 2025, ran in auto mode (Section 4.5).

3.4 Memory and retrieval over past sessions

The motivation for memory is that a lab agent should learn from earlier attempts: which windows showed resonances, which candidates were dead, which parameters failed. QSA's retrieval went through two designs (details in Appendix A). In **May 2025** (commits `2933be4`, `7943e46`), each session is saved on exit as one all-MiniLM-L6-v2 [33] vector with its text, and on every turn the three stored lines closest to the latest operator message are appended to the prompt, with plot file names. In **June 2025** (commits `339fbed`, `a233b2b`, `f5d7c42`), sessions are embedded in overlapping chunks of 10 turns, and retrieval becomes an action, `rag_search`, that the agent invokes "when you feel stuck, need to learn from past experience". Each chunk carries the names of the plots it refers to, and the `vision` action accepts paths into earlier run folders, so the agent can reopen a historical plot. In the committed June code, saving on exit fails because the chunking code uses a variable before assigning it, so no June session was embedded; the public release fixes this. The corpus is the lab's own past agent sessions; no external documentation is indexed.

3.5 Evolution of the agent

From the log formats and commit history we identify six versions (Figure 1, bottom; Appendix A): **v0** (February–March, gated, several actions per reply), **v1**/**v1b** (April refactor), **v2** (2 May retrieval prototype, development machine), **v3** (14 May lab version, automatic retrieval, no logged gate) and **v4** (June code, run on the hardware once, on 16 June).

4 Deployment and results

4.1 Overview

We analysed 31 logs from branch `june-14-system-prompt-fixes` of the repository: two root logs from 18–19 February and 29 run folders from 28 February – 16 June 2025. They contain 36 sessions, where a session is a part of a log with no gap longer than 3 h. A session counts as reaching the hardware if its script output contains instrument-driver messages or a "saved data" line. A measurement counts as *completed* only if such a line appears inside logged script output; a line inside an LLM reply does not count. This definition matters. Counting every "saved data" line in the logs would give 79 for February–May, because the LLM invented 7 such lines (Section 4.4); the correct count is 72, or 73 including the 16 June scan. Table 1 and Figure 1 summarise the record, and Table 2 gives agent-level statistics.

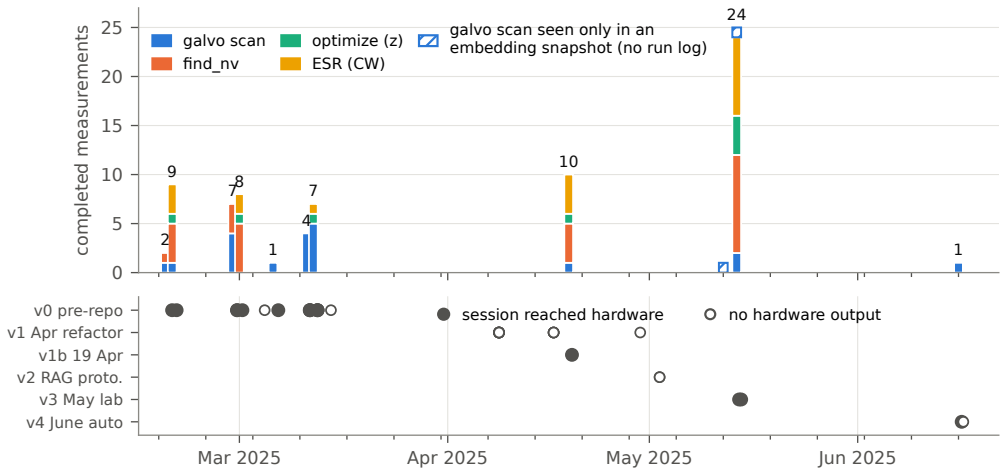


Figure 1: Deployment timeline. **Top:** completed hardware measurements per day, stacked by type (73 in total). Hatched: galvo scans recorded only in embedding snapshots (12 May; 14 May 07:58), not counted in the total. **Bottom:** every agent session by agent version; filled markers reached the hardware. The first hardware session began on 18 February 2025 at 23:43.

Most of the instrument time went into ODMR. The median CW sweep took 14.1 min, while galvo, `find_nv` and optimize runs took under a minute each. The overhead of the agent was small compared with the instrument time: the median LLM latency was 8 s and the median operator reply 16 s. The richest sessions were 19 April (195 min, 40 LLM calls, 10 measurements) and the morning of 14 May (204 min, 51 LLM calls, 20 measurements). April produced little hardware data: apart from 19 April, the April sessions were short development runs that failed to launch scripts.

4.2 Case study: 14 May 2025

The operator opened the session with "characterize all the NV in the field of view". The agent read the default galvo configuration, wrote its own, and ran the scan (49 s; Figure 2a). It then requested a vision analysis of the scan, but in the same reply it had already written four coordinates. The logged vision result repeats that list word for word, and none of the four matches a spot. The operator read six candidates off the plot instead and told the agent to "remember the locations". The agent kept this list across the 3.4-hour session and worked through it:

- **Candidate 1.** Two `find_nv` runs (the second re-centred by the operator), two z-optimizations (maximum 9.24 kcps; Figure 2c) and two ESR sweeps (2.82–2.92 and 2.72–2.82 GHz at −8 dBm), with no resonance.
- **Candidate 2.** Three `find_nv` runs, one optimize and three sweeps, including one at 0 dBm, again with no resonance.

Asked "can you look at the historical attempts and tell me why we are not seeing ESR", the agent summarised both candidates from the transcript. It listed possible causes in three groups: *microwave delivery* (antenna or wire position, power reaching the NV), *NV properties* (misalignment with the field, a different charge state, different resonance frequencies) and *equipment status* (recurring PulseBlaster errors in the logs). The operator checked the microwave delivery physically and moved on. On candidate 3 the operator re-centred `find_nv` and, after one empty sweep, asked for a 2.85–2.89 GHz window at 0 dBm. The agent wrote a configuration with exactly these values. The sweep showed two dips (Figure 2d), and the operator logged "good we see ESR". The session continued to candidate 4 (two `find_nv` runs) before it ended. In a short afternoon session the same day, the agent took a configuration key (`esr_RnS`) for a script name. The parser rejected the command, and the agent corrected it on the next turn (39 s from the error to the corrected run; Appendix F).

Table 2: Agent-level statistics over all 31 logs. Script wall time includes instrument time and Python start-up. Cycle time is the interval between consecutive completed measurements within a session.

Quantity	Value
Sessions / reaching hardware / hardware days	36 / 16 / 10
Wall-clock time of hardware sessions	976 min
Instrument time (of which CW ESR)	311 min (276 min)
Median script wall time: galvo / `find_nv` / optimize / ESR	49 s / 20 s / 25.5 s / 847 s
Median cycle time between measurements ($n = 57$)	9.3 min (range 0.8–68.3)
LLM calls; median LLM latency	321; 8 s
Median operator reply latency ($n = 463$)	16 s
Executed actions: message / read / write / run / vision	248 / 83 / 71 / 92 / 48
Actions refused at the permission prompt: write / run / vision[a]	11 / 8 / 5
Permission answers: yes / no / other	158 / 22 / 2
Run commands: completed / failed	73 / 19
Operator guidance messages (in hardware sessions)	174 (133)
Parametric guidance reflected exactly in the next config	22 / 22
LLM replies containing invented `User:`/`Assistant:` turns	32 / 321 (10.0%)
Logged retrieval queries: 2 May (development machine) / 16 June (hardware)	5 / 1

[a] 24 refusals: the 22 "no" answers, plus a "no" followed by an instruction and a mistyped "ye"; the gate accepted only an exact "yes".

The candidate-3 spectrum. No spectrum in the deployment was fitted at the time. For this report we fitted the candidate-3 sweep with weighted Lorentzian models (Appendix Figure 4). Two dips with a shared width are preferred over one dip ($\Delta\mathrm{AIC} = 10.5$; $\chi^2/\mathrm{dof} = 27.6/34$ versus $42.1/36$) and over a flat line. The dips sit at $f_1 = 2863.1 \pm 0.8\,\mathrm{MHz}$ and $f_2 = 2877.5 \pm 0.5\,\mathrm{MHz}$, with contrasts of $2.4 \pm 0.6\%$ and $3.9 \pm 0.6\%$. Their mid-point (2870.3 MHz) is consistent with D, and their splitting is $\Delta f = 14.4 \pm 0.9\,\mathrm{MHz}$ (statistical; modelling choices give 14.4–15.7 MHz). If interpreted as a Zeeman splitting, it would correspond to $B_\parallel = \Delta f / 2\gamma \approx 2.6\,\mathrm{G}$. The bias field on 14 May is not known, however, and strain or other local effects could also split the resonance, so we do not attribute the splitting to an applied field. This is a single 175 s sweep with a few-percent contrast; it shows that the chain reached a resonance, not a precision measurement.

Division of labour. The agent wrote configurations, invoked scripts, kept the candidate list, summarised earlier attempts and listed hypotheses. The operator chose the candidates, re-centred scans, chose the frequency windows, and checked the microwave delivery physically.

4.3 Guidance and troubleshooting

Parametric guidance was followed exactly. We tested every operator message that named coordinates, a frequency range or a power against the next configuration the agent wrote. In 22 of 22 cases the configuration contained exactly the requested values: 11 `find_nv` centres, 10 ESR windows and one power change.

Domain knowledge was adopted within a session but not across sessions. On 19 April the operator tagged messages `#rule`, `#experience` and `#suggest`, for example that an ESR count rate far below the `find_nv` rate suggests a quenched NV. The agent restated each point in its next reasoning block and acted on it; after the quenching hint it compared 11.5 with 2.9 kcps and moved to the next candidate (Appendix F). None of these lessons reappears in any later log. The 19 April session predates the embedding store, and on 14 May the operator, not the agent, made the equivalent call.

Script failures. Of 92 run commands, 73 completed and 19 failed; the agent recovered from the 14 May parse error, but not from most other failures (Appendix B).

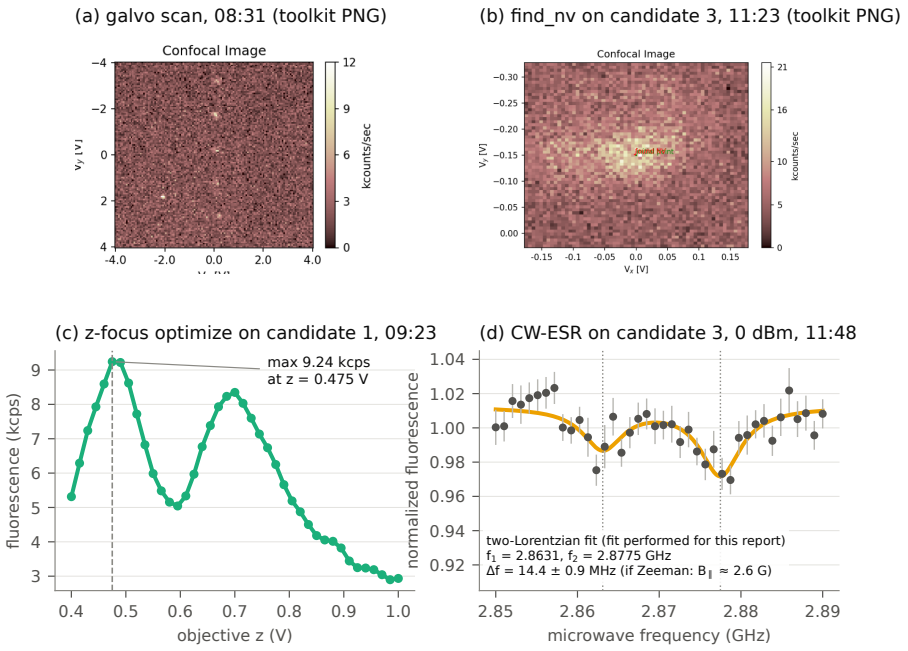


Figure 2: The 14 May 2025 chain (run `run_20250514_082859`). (a) Galvo scan at 08:31, as rendered by the lab toolkit (its x-axis label is clipped in the original file). (b) `find_nv` on candidate 3 at 11:23, re-centred by the operator (the overlapping red and green labels are in the toolkit's original). (c) z-focus optimize at 09:23 on candidate 1; candidate 3 went from `find_nv` straight to ESR. (d) CW ODMR on candidate 3 (2.85–2.89 GHz, 40 points, 0 dBm, 50 averages, 175 s) with a two-Lorentzian fit that we performed for this report; the toolkit fitted nothing. Data recovered from git commit `b316b58`.

4.4 Failure analysis

Invented tool output. Because the history was serialized as a flat `User:`/`Assistant:` transcript, the model sometimes continued the transcript on its own. 29 of the 311 February–May replies (9.3%) contain invented `User:` or `Assistant:` lines, and 32 of 321 (10.0%) with 16 June. Five February–May replies (in v0 and v1b) invent script output, with seven fabricated "saved data" lines; three replies on 16 June add three more. In one reply on 11 March the model invented an operator permission, an optimize run and an "optimized position". The real optimize run failed, yet the agent planned the next ESR measurement at the invented position; the operator refused the write, the run and the plot analysis. The parser executed every `<action>` block in a reply, including blocks inside the invented continuation. The permission gate therefore did real work: 19 of the 22 "no" answers in the logs were given to replies that contained invented turns. Counting such lines naively would inflate the February–May measurement count from 72 to 79. In the 14 May version, which had no gate, the three affected replies only append an invented "`User: yes`" and contain no invented results. The June code executes only the first action of each reply, so invented actions no longer run, but it keeps the flat transcript; the 16 June test below shows that this is not enough without a human in the loop.

Vision was unreliable for coordinates. The agent made 48 vision calls, 47 of them in February–May. Whenever candidate coordinates were needed, the operator either rejected the agent's reading (28 February: "you did it wrong, read the plot again") or supplied the coordinates (19 April: "Please use the list that I read visually"; 14 May: six typed coordinates). On 14 May the agent used vision twice in 60 calls.

Low ODMR contrast and no positive control. The clearest resonance in the record is the 14 May candidate-3 spectrum, at a few-percent contrast. On the same day, five sweeps on candidates 1 and 2 showed none. The agent could hypothesize causes but had no instrument-level test to distinguish them; the decisive

checks (MW delivery, quenching) came from the operator.

Retrieval did not visibly matter. In February–May the only logged retrievals are five queries on 2 May on a development machine. They returned single-line snippets (similarity 0.38–0.44) from a test conversation, and the agent replied that it saw no relevant earlier settings. On 14 May every turn logged "Building prompt with RAG context", but no retrieval result appears in the logs, and no reply refers to an earlier session. The one retrieval on the hardware, on 16 June, returned nothing relevant (Section 4.5). Cross-session improvements that did happen (`python`→`py`, the `-output-dir` fix on 14 May) were made by humans in code.

4.5 Post-deployment auto-mode test, 16 June 2025

One session ran the June code (v4) on the hardware, in auto mode (`run_20250616_112227`; 11:22–11:26; 10 LLM calls after the single prompt "characterize NV for me"). The agent read the default galvo configuration, wrote its own and ran the scan. The script loaded the PulseBlaster driver, which reported a communication error, and saved a scan after 63 s; the agent sent the plot to vision. This is the session's one completed measurement. In the next three replies the model continued the transcript with invented `Assistant:` turns containing fabricated script output: a `find_nv` run, an optimization ("Optimized coordinates: X=-3.92, Y=1.05, Z=0.15") and an ESR sweep ending in "ESR transitions found at: 2.8671 GHz, 2.8732 GHz", followed by an invented summary that an NV had been "successfully characterized". Because v4 executes only the first action of a reply, none of these invented actions ran: `find_nv` was never run, and the optimize and ESR configurations the model had "written" are not in the run folder. The real optimize and ESR commands that followed pointed to these missing files and failed with exit status 1. The agent then called `rag_search` for "successful ESR measurement configurations"; its query already contained the invented ESR result, and neither returned chunk (similarity 0.57 and 0.54) concerned ESR. The last reply reasoned "We had some success with the earlier steps (galvo_scan, find_nv, optimize)" and drafted a message that reported the invented optimized coordinates as a result and asked the operator for ESR settings; the log ends there. Unsupervised, the agent believed its own invented results. In the gated February–April versions the operator refused actions from such replies; here nobody was in the loop, and only the log revealed the problem.

5 Discussion

What worked. Without hardware-specific training, a general-purpose LLM operated the measurement chain of a real confocal NV setup through a narrow, text-only interface: JSON configurations, four scripts, and plots. It translated every parametric operator instruction into the requested configuration (22 of 22), kept track of a six-candidate list over 3.4 hours, recovered from a script-name error within one turn, and produced reasonable hypotheses when a measurement failed. Because the agent could act only through parameter values passed to the lab's own toolkit scripts, its mistakes stayed within what an ordinary user of those scripts could do. The detailed logs made every measurement auditable after the fact, which is what allowed us to find the fabricated output after the fact.

What did not. The physics outcome was modest: few-percent ODMR contrast on one candidate in the best session, with no fit at the time. The agent depended on the operator's domain priors for every scientifically important decision: which spots were NVs, where to centre, which frequency window to try, when an NV was quenched, and whether the microwaves reached the sample. Reading coordinates from plots, which the workflow needs, failed. QSA ran only CW ODMR, so it never reached the pulsed measurements, where automation would help most. Learning across sessions remained a design rather than an observed effect. Left unsupervised in June, the agent reported invented results as its own progress.

Lessons for agents in quantum labs.

1. *Use structured chat messages, not a flat transcript.* A flat `User:`/`Assistant:` prompt invites the model to write the operator's next turn and the instrument's output, and an action parser that executes every block turns that into real actions.

2. *Gate actions, and log the gate.* The gate gave the operator a point at which to refuse actions from replies with invented turns, and 19 of 22 refusals did so; when the gate disappeared on 14 May, so did the record of what was approved, and in the ungated auto mode of 16 June invented results went unchallenged.
3. *Executing one action per turn is not enough.* It stopped invented actions from running, but not invented results from entering the transcript, the retrieval query and the agent's own summary. Tool output should be marked by the harness, for example in separate tool-result messages, so that the model cannot forge it.
4. *Count only what the instrument wrote.* Any summary statistic must be computed from tool output, not from text the model produced.
5. *Give the agent numerical tools for numerical reading.* Peak finding, fitting and spot detection should be tools; asking a vision model for coordinates was not reliable.
6. *Make lessons persistent and explicit.* The operator's tagged rules (`#rule`, `#experience`) were exactly the kind of knowledge that should persist across sessions. We expect that a small store of explicit lab rules would serve better than retrieval over raw conversations.

How the field has moved. Since early 2025 LLM agents have formalized per-action authorization [31], kept persistent lab memories [25, 30], moved toward rule engines in which the LLM handles only language and exceptions [10], and, on NV centers, run pulsed protocols autonomously for many hours [13]. QSA is an early, mostly human-supervised point on this trajectory, and we offer its record mainly as a dated, honest account of what such an agent could and could not do in early 2025.

6 Limitations

- **Scale.** This is a deployment record, not a controlled study. It covers 16 hardware sessions on one setup, with one main operator pair, and has no baseline (for example, the same tasks run by hand).
- **Provenance.** The exact code of several versions is not in git: the February–March code predates the first commit, 19 April used an uncommitted variant, and the 14 May code differs from the nearest commit. The models used per session rest on the authors' record; the logs do not name them.
- **Missing logs.** Some sessions (12 May; 14 May 07:56) are known only from embedding snapshots.
- **Physics.** Only CW ODMR was run, and the one fitted spectrum is a single low-contrast sweep whose splitting cannot be attributed to a known field.
- **Barely tested features.** The June features ran on the hardware only in the four-minute auto-mode test of 16 June 2025; no logged hardware session used assistant mode or reopened a historical plot.
- **Priority.** Our priority statement compares an internal log date with other groups' public posting dates, and our literature search relied on arXiv and Crossref because Semantic Scholar was unavailable; work not on arXiv may be missed.

Code and data availability

The code, the configurations and the run folders with all agent logs analysed here are released at `https://github.com/zechenzhangAGI/quantum-sensing-agent` under the MIT License. The measurement data used in Figures 2 and 4 are in `examples/2025-05-14-nv-odmr/` of the release; they are byte-identical to files recovered from the development history. The analysis scripts that reproduce every number and figure in this report are in `analysis/` of the release. Commit hashes in this report refer to the development history and are given with their dates.

Author contributions

Z.Z. conceived and led the project, developed the agent, and performed the experiments with R.X. R.X. operated the setup with Z.Z. and provided NV/ODMR expertise. J.F. co-developed the early agent code and the first retrieval prototype. A.Y. supervised the work and provided the experimental platform.

Acknowledgments

We thank the repository contributors rholanx and SamC1249. Some June 2025 commits in the repository were authored with an AI coding assistant (google-labs-jules[bot]). AI assistants were used to help analyze logs and prepare this manuscript; the authors take full responsibility for its content.

References

[1] Brandon Bauer, Matthew Whalen, Kenji Watanabe, Takashi Taniguchi, John Q. Xiao, Tao E. Li, and Wenjin Zhao. Autonomous Quantum Transport Measurements of 2D Semiconductors by an AI Agent. *arXiv preprint arXiv:2609.26661*, 2026.

[2] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023. doi: 10.1038/s41586-023-06792-0. arXiv preprint titled "Emergent autonomous scientific research capabilities of large language models".

[3] C. Bonato, M. S. Blok, H. T. Dinani, D. W. Berry, M. L. Markham, D. J. Twitchen, and R. Hanson. Optimized quantum sensing with a single electron spin using real-time adaptive measurements. *Nature Nanotechnology*, 11:247–252, 2015. doi: 10.1038/nnano.2015.261.

[4] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6:525–535, 2024. doi: 10.1038/s42256-024-00832-8. arXiv preprint titled "ChemCrow: Augmenting large-language models with chemistry tools".

[5] Shuxiang Cao, Zijian Zhang, Mohammed Alghadeer, Simone D. Fasciati, Michele Piscitelli, Mustafa Bakr, Peter Leek, and Alán Aspuru-Guzik. Automating quantum computing laboratory experiments with an agent-based AI framework. *Patterns*, 6:101372, 2025. doi: 10.1016/j.patter.2025.101372. arXiv preprint titled "Agents for self-driving laboratories applied to quantum computing".

[6] Shuxiang Cao, Zijian Zhang, Abhishek Agarwal, Grace Bratrud, Niyaz R. Beysengulov, Daniel C. Cole, Alejandro Gómez Frieiro, Elena O. Glen, Hao Hsu, Gang Huang, Raymond Jow, Greshma Shaji, Tom Lubowe, Ligeng Zhu, Luis Mantilla Calderón, Nicola Pancotti, Joel Pendleton, Brandon Severin, Charles Etienne Staub, Sara Sussman, Antti Vepsäläinen, Neel Rajeshbhai Vora, Yilun Xu, Varinia Bernales, Daniel Bowring, Elica Kyoseva, Ivan Rungger, Giulia Semeghini, Sam Stanwyck, Timothy Costa, Alán Aspuru-Guzik, and Krysta Svore. QCalEval: Benchmarking Vision-Language Models for Quantum Calibration Plot Understanding. *arXiv preprint arXiv:2604.25884*, 2026.

[7] Constantin Dalyac, Alexandre Dauphin, Loïc Henriet, and Christophe Jurczak. Lowering the implementation barrier of neutral-atom quantum computing with agentic workflows. *arXiv preprint arXiv:2607.25834*, 2026.

[8] Marcus W. Doherty, Neil B. Manson, Paul Delaney, Fedor Jelezko, Jörg Wrachtrup, and Lloyd C. L. Hollenberg. The nitrogen-vacancy colour centre in diamond. *Physics Reports*, 528(1):1–45, 2013. doi: 10.1016/j.physrep.2013.02.001.

[9] Sergey Dushenko, Kapildeb Ambal, and Robert D. McMichael. Sequential Bayesian Experiment Design for Optically Detected Magnetic Resonance of Nitrogen-Vacancy Centers. *Physical Review Applied*, 14: 054036, 2020. doi: 10.1103/PhysRevApplied.14.054036.

[10] Zhiqiang Fan, Haoran He, Ping Lv, Junchao Wang, Yaqiang Sun, Chenhui Wang, Hanshi Zhao, Geyuyan Ma, Haoran Yang, Pengyu Han, Xiangdong Meng, Lixin Wang, Feng Yue, Weilong Wang, and Zheng Shan. QMClaw: A Scalable General-purpose Framework for Quantum Measurement and Control. *arXiv preprint arXiv:2609.04674*, 2026.

[11] Thomas Palmeira Ferraz, Romain Deffayet, Vassilina Nikoulina, Hervé Déjean, and Stéphane Clinchant. Retrieval-Augmented LLM Agents: Learning to Learn from Experience. *arXiv preprint arXiv:2603.18272*, 2026.

[12] Thorsten Hellert, Drew Bertwistle, Simon C. Leemann, Antonin Sulc, and Marco Venturini. Agentic artificial intelligence for multistage physics experiments at a large-scale user facility particle accelerator. *Physical Review Research*, 8:L012017, 2026. doi: 10.1103/jtqy-9jz1.

[13] Takuya Isogawa, Ryotaro Okabe, Nutdech Phadetsuwannukun, Mingda Li, and Paola Cappellaro. Agentic AI for Scientific Reasoning in Autonomous Quantum Sensing Experiments. *arXiv preprint arXiv:2607.25145*, 2026.

[14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv preprint arXiv:2005.11401*, 2020.

[15] Shiheng Li, Jacob M. Miller, Phoebe J. Lee, Gustav Andersson, Christopher R. Conner, Yash J. Joshi, Bayan Karimi, Amber M. King, Howard L. Malc, Harsh Mishra, Hong Qiao, Minseok Ryu, Xuntao Wu, Siyuan Xing, Haoxiong Yan, Jian Shi, and Andrew N. Cleland. Large Language Model-Assisted Superconducting Qubit Experiments. *arXiv preprint arXiv:2603.08801*, 2026.

[16] Yongtao Liu, Marti Checa, and Rama K. Vasudevan. Synergizing Human Expertise and AI Efficiency with Language Model for Microscopy Operation and Automated Experiment Design. *arXiv preprint arXiv:2401.13803*, 2024.

[17] Ryan Lopez, Sachin Vaidya, Seou Choi, Serena Landers, and Marin Soljačić. Vision-Language Agents for Active Perception in Optics Laboratories. *arXiv preprint arXiv:2609.32918*, 2026.

[18] Indrajeet Mandal, Jitendra Soni, Mohd Zaki, Morten M. Smedskjaer, Katrin Wondraczek, Lothar Wondraczek, Nitya Nand Gosvami, and N. M. Anoop Krishnan. Evaluating large language model agents for automation of atomic force microscopy. *Nature Communications*, 16:9104, 2025. doi: 10.1038/s41467-025-64105-7. arXiv preprint titled "Autonomous Microscopy Experiments through Large Language Model Agents".

[19] Robert D. McMichael and Sean M. Blakley. Simplified Algorithms for Adaptive Experiment Design in Parameter Estimation. *Physical Review Applied*, 18:054001, 2022. doi: 10.1103/PhysRevApplied.18.054001.

[20] Robert D. McMichael, Sergey Dushenko, and Sean M. Blakley. Sequential Bayesian experiment design for adaptive Ramsey sequence measurements. *Journal of Applied Physics*, 130:144401, 2021. doi: 10.1063/5.0055630.

[21] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. *arXiv preprint arXiv:2304.03442*, 2023.

[22] Swetapadma Sahoo, Junyue Jiang, Jaden Li, Kieran Loehr, Chad E. Germany, Jincheng Zhou, Bryan K. Clark, and Simeon I. Bogdanov. FiND: Few-shot three-dimensional image-free confocal focusing on point-like emitters. *arXiv preprint arXiv:2311.06479*, 2023.

[23] R. Santagati, A. A. Gentile, S. Knauer, S. Schmitt, S. Paesani, C. Granade, N. Wiebe, C. Osterkamp, L. P. McGuinness, J. Wang, M. G. Thompson, J. G. Rarity, F. Jelezko, and A. Laing. Magnetic-Field Learning Using a Single Electronic Spin in Diamond with One-Photon Readout at Room Temperature.

Physical Review X, 9:021019, 2019. doi: 10.1103/PhysRevX.9.021019. arXiv preprint titled "Magnetic-field-learning using a single electronic spin in diamond with one-photon-readout at room temperature".

[24] Jonas Schuff, Miguel J. Carballido, Madeleine Kotzagiannidis, Juan Carlos Calvo, Marco Caselli, Jacob Rawling, David L. Craig, Barnaby van Straaten, Brandon Severin, Federico Fedele, Simon Svab, Pierre Chevalier Kwon, Rafael S. Eggli, Taras Patlatiuk, Nathan Korda, Dominik M. Zumbühl, and Natalia Ares. Fully autonomous tuning of a spin qubit. *Nature Electronics*, 9:304–313, 2026. doi: 10.1038/s41928-025-01562-4.

[25] Rong Sha, Binglin Wang, Jun Yang, Xiaoxiao Ma, Chengkun Wu, Liang Yan, Chao Zhou, Jixun Liu, Guochao Wang, Shuhua Yan, and Lingxiao Zhu. LLM-based Multi-Agent Copilot for Quantum Sensor. *arXiv preprint arXiv:2508.05421*, 2025.

[26] Chao Shang and Gregory D. Fuchs. Fast Single Nitrogen-Vacancy Center Ramsey Characterization using a Physics-Informed Neural Network. *arXiv preprint arXiv:2603.14144*, 2026.

[27] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv preprint arXiv:2303.11366*, 2023.

[28] Gary Tom, Stefan P. Schmid, Sterling G. Baird, Yang Cao, Kourosh Darvish, Han Hao, Stanley Lo, Sergio Pablo-García, Ella M. Rajaonson, Marta Skreta, Naruki Yoshikawa, Samantha Corapi, Gun Deniz Akkoc, Felix Strieth-Kalthoff, Martin Seifrid, and Alán Aspuru-Guzik. Self-Driving Laboratories for Chemistry and Materials Science. *Chemical Reviews*, 124:9633–9732, 2024. doi: 10.1021/acs.chemrev.4c00055.

[29] Shiekh Zia Uddin, Sachin Vaidya, Shrish Choudhary, Zhuo Chen, Raafat K. Salib, Luke Huang, Dirk R. Englund, and Marin Soljačić. AI-Driven Robotics for Optics. *arXiv preprint arXiv:2505.17985*, 2025.

[30] Aikaterini Vriza, Michael H. Prince, Tao Zhou, Henry Chan, and Mathew J. Cherukara. Operating advanced scientific instruments with AI agents that learn on the job. *arXiv preprint arXiv:2509.00098*, 2025.

[31] Duanyang Wang, Lu Qi, Yuanheng Xie, Norbert M. Linke, and Kenneth R. Brown. A hardware-safety-gated system for LLM-written native ARTIQ control code on a trapped-ion platform. *arXiv preprint arXiv:2606.27231*, 2026.

[32] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*, 2023.

[33] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. *arXiv preprint arXiv:2002.10957*, 2020.

[34] Huikai Xu, Jiaxiu Han, Shigang Ou, Cheng Ye, Zisong Shen, Jing Gao, Yijia Wang, Tianrui Che, Yu Song, Weiyang Liu, Lei Wang, Lin-Feng Zhang, Pan Zhang, and Hai-Feng Yu. Vibe Calibration: Autonomous Bring-up of a 112-Qubit Superconducting Quantum Processor by a Skill-Orchestrating Language Agent. *arXiv preprint arXiv:2606.22376*, 2026.

[35] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM Agents Are Experiential Learners. *arXiv preprint arXiv:2308.10144*, 2023.

A Supplementary system details

Hardware and scripts. An NI 6353 DAQ steers the galvo mirrors and the z-piezo with analog outputs and counts photons; a Rohde & Schwarz generator, addressed over VISA, provides the microwaves; a SpinCore PulseBlaster is loaded by every script. Positions are given as control voltages; the repository contains no voltage-to-micron calibration. `galvo_scan` rasters the field of view (default 126×126 points over ± 4 V, 2 ms per point); bright spots are NV candidates. `find_nv` takes a 61×61 zoomed scan over a 0.35 V square around a chosen point, fits a 2D Gaussian and moves the beam to the fitted maximum. `optimize` maximizes the count rate with 1D sweeps (default: z only, 41 points over 0.6 V). `ESR` sweeps the microwave frequency from `freq_start` to `freq_stop` at power `power_out`, averaging `esr_avg` times and normalizing to a reference (default 2.90–2.95 GHz, 100 points, -8 dBm).

Prompting. The system instruction states the overall goal ("systematically measure the ESR of multiple NV centers in a diamond chip"), describes each script's inputs, outputs and caveats, gives the typical workflow, lists the default configuration paths, and fixes the run-command format. It tells the agent to read the default configuration first and then change parameter values only. Until June the history was serialized into a single flat `User:/Assistant:` transcript; Section 4.4 shows why this mattered.

Vision. The agent reads every plot type through the `vision` action, which sends the PNG together with recent context to the vision model. For galvo scans the prompt asks it to read out the centre coordinates of bright spots.

Retrieval details.

- **May 2025 (commits `2933be4`, `7943e46`).** On exit, the whole session is saved as one 384-dimensional all-MiniLM-L6-v2 vector together with its text. On every turn the latest operator message is used as a query. Stored sessions are split into lines and re-embedded, and the top three lines (FAISS, L2 distance) are appended as "relevant context from previous conversations", together with plot file names.
- **June 2025 (commits `339fbed`, `a233b2b`, `f5d7c42`).** Chunks of 10 turns with an overlap of 5. `rag_search` uses a contextualized query and returns the top two chunks by cosine similarity in a delimited "past experiments" block. The save-on-exit bug dates from `34b0137` (14 June 2025).
- **Unmerged.** A separate branch (`dd6fe22`, 31 May) added distinct error and general memories that are written automatically when a script fails.

Agent versions. **v0** (18 Feb – 14 Mar) predates the repository; its first commit (3 April 2025) imports these logs. It allows several actions per reply, all gated. **v1** (April) refactors the code into per-project folders, and **v1b** (19 April) is an uncommitted local variant of it. **v2** (2 May) is the retrieval prototype, run on a development machine, and **v3** (14 May) is the May lab version with automatic retrieval and no logged gate. The June 2025 code, **v4** (assistant and auto modes, one action per turn, `rag_search`, historical plot analysis), is the most developed; it ran on the hardware once, in a four-minute auto-mode test on 16 June 2025.

B Supplementary results

19 April tagged guidance. On 19 April the operator tagged messages `#rule`, `#experience` and `#suggest`. For example:

- `#experience`: single-NV contrast is "a few percent (bad) to 20% (best)", followed by a suggestion to scan 2.80–2.84 GHz.
- `#suggest`: when the count rate during ESR is far below the `find_nv` count rate, the NV may be quenched and one should move on.

The agent restated each point in its next reasoning block and acted on it. After the `#experience` message it wrote a 2.80–2.84 GHz configuration. After the `#suggest` message it compared 11.5 with 2.9 kcps, abandoned that NV, and moved to the next candidate (Appendix F). None of these lessons reappears in any later log. The 19 April session predates the embedding store, and on 14 May the operator, not the agent, made the equivalent call.

Script failures. Of 92 run commands, 73 completed and 19 failed. The agent recovered from the 14 May parse error. It did not recover from the five "`python`" invocations on 19 February (the next session used `py`, a change made in code between sessions), nor from the exit-127 failures on 8 April, which are consistent with running the Windows launcher on a development machine. The two failures on 16 June are discussed in Section 4.5.

Messages ahead of results. Before the June one-action rule, a single reply could contain both a `run` and a `message`. On 14 May the message announcing "I've started the ESR measurement" was composed before the run's result and so reported success for a command that had failed.

C Extended related work

Other laboratory agents. Self-driving laboratories are reviewed in [28]. Since 2025, agents have run an X-ray nanoprobe beamline with human-in-the-loop corrections stored in a vector database [30], multistage experiments at an accelerator facility [12], and multi-day cryogenic transport measurements [1].

Other quantum platforms. After k-agents [5], LLM agents have addressed superconducting qubits [10, 15, 34], trapped ions with per-action authorization tokens [31], neutral atoms [7] and optics [17, 29], and a benchmark tests vision-language models on reading calibration plots [6]. Classical ML has also automated quantum devices without language models, for example tuning of a semiconductor spin qubit [24].

Classical automation of NV experiments. Adaptive Bayesian methods choose measurement settings on the fly for single-NV magnetometry [3, 23], CW ODMR [9] and Ramsey measurements [20], with simplified adaptive-design algorithms in [19]; FiND [22] focuses on point emitters without imaging; neural networks speed up Ramsey characterization [26]. These methods optimize inside a fixed protocol; a Bayesian ODMR routine could serve as one of QSA's tools.

D Abridged system prompt

The assistant-mode system instruction in `agent/agent.py` at commit `34b0137` (branch `june-14-system-prompt-fixes`), abridged; "[...]" marks omissions. The February version (`17c17e1`) has the same structure but allows "zero or more `<action>` blocks" per reply and has no `rag_search`.

```
You are the NVExperimentAgent operating in ASSISTANT MODE, a specialized
assistant for nitrogen-vacancy (NV) center experiments in diamond chips.

OVERALL GOAL: Your primary objective is to utilize the available experimental
scripts to systematically measure the ESR (Electron Spin Resonance) of multiple
NV centers in a diamond chip. [...]

ASSISTANT MODE BEHAVIOR: [...] You must ask for permission before taking any
actions that modify files or run experiments. [...]

AVAILABLE EXPERIMENTAL SCRIPTS:
1. galvo_scan: Performs a coarse scan of the entire diamond chip [...]
   - ANALYSIS: Bright spots indicate potential NV centers, but results should
     be verified with find_nv
2. find_nv: [...] High-resolution brightness map with 2D Gaussian fit results
3. optimize: Performs 1D scans in X, Y, and Z directions [...]
4. ESR: [...] Fitted Gaussian minima (red labels) indicate ESR resonance
```

```
   frequencies, typically around 2.87 GHz for NV centers

TYPICAL WORKFLOW:
- Start with galvo_scan to map the chip and identify NV locations
- Use find_nv to precisely locate individual NVs from the coarse scan
- Run optimize to achieve optimal focus for measurements
- Perform ESR measurements on the located and optimized NV centers
[...]
2) Actions:
   - IMPORTANT: You may produce exactly ONE <action> block per response
   - The "type" must be one of: "message", "read", "write", "run", "vision",
     or "rag_search".
3) Security & Directory Rules:
   - Read Access: Only from the configs\ or data\ directories.
   - Write Access: Only to the configs\ or data\ directories.
   - Run Access: Only scripts in the scripts\ directory.
[...]
5) Run Command Options: [...]
     py <scripts_dir>\<script_name>.py --config <config_file>
        --output-dir projects\NVExperiment\runs\run_<TIMESTAMP>\data\
6) Vision Option: [...] For GalvoScan_plot.png, NVs are associated with large
   bright dots; estimate and read out the center coordinates of bright dots
   for subsequent steps.
7) Configuration Management & Usage Flow:
   - CRITICAL: When writing new configs, you MUST maintain the EXACT same
     structure and format as the default config
   - CRITICAL: Only modify parameter values, never change keys, structure,
     or data types
   - Always read the appropriate default config first [...]
8) Behavior & Permissions:
   - When you <write> a file, ask the user permission. If denied, do not write.
   - When you <run> or <vision> a command, ask the user permission. [...]
11) RAG Search Tool:
   - When you feel stuck, need to learn from past experience, or believe
     relevant information exists in previous conversations, use the
     "rag_search" tool [...]
   - Historical Plot Analysis: When RAG search returns references to
     historical plots [...], you can analyze those plots using vision action
     with the full historical path
```

E Configuration schema

Default ESR configuration (`projects/NVExperiment/configs/default_esr_config.json`); the agent may change values under `settings` only. The instrument address and the lab-PC file path are redacted here. The other three scripts use the same layout (`scripts` → toolkit script name → `class`, `filepath`, `instruments`, `settings`).

```
{ "scripts": { "esr_RnS": {
    "class": "ESR",
    "filepath": "<lab PC path>\\b26_toolkit\\scripts\\esr_RnS.py",
    "info": "ESR measurement script using R&S microwave generator",
    "instruments": {
      "microwave_generator": { "class": "R8SMicrowaveGenerator",
                               "settings": { "VISA_address": "<redacted>" } },
      "NI6353": { "class": "NI6353", "settings": {} },
      "PB":     { "class": "LISE607RTPulseBlaster", "settings": {} } },
    "settings": {
      "power_out": -8.0, "esr_avg": 100,
      "freq_start": 2.9e9, "freq_stop": 2.95e9, "range_type": "start_stop",
      "freq_points": 100, "integration_time": 0.05, "num_samps_per_pt": 100,
      "mw_generator_switching_time": 0.01, "turn_off_after": true,
      "norm_to_ref": true, "save_full_esr": true, "daq_type": "PCI",
      "fit_constants": { "num_of_peaks": -1, "minimum_counts": 0.9,
                         "contrast_factor": 3.0,
```

```
                        "zfs": 2.87e9, "gama": 2.8028e6 },
      "track_laser_power": { "on/off": false, "ai_channel": "ai4" } } } } }
```

For the candidate-3 sweep on 14 May the agent wrote `esr_config_NV3_modified2.json`, changing exactly `freq_start` = 2.85e9, `freq_stop` = 2.89e9, `freq_points` = 40, `esr_avg` = 50 and `power_out` = 0.0.

F Additional log excerpts

Verbatim log lines; the number before each line is its line number in the log file, and "[...]" marks shortened lines.

14 May 2025: operator steering and the agent's diagnosis (`run_20250514_082859`).

```
1:    [2025-05-14 08:29:41] USER: characterize all the NV in the field of view
157:  [2025-05-14 09:01:16] USER: i read the spot as (0.2,-3.3),(0.2,-1.8),(0.2,-0.3),(0.2,1.4),(0.2,2.8),(2,1.9)
205:  [2025-05-14 09:02:46] USER: remember the locations, you need to run find NV on the first NV
1056: [2025-05-14 10:03:29] USER: we may see or didn't see ESR, could be this NV having problem or the MW delivery has
↪  problem. Let us move to the next NV on our list
2027: [2025-05-14 10:57:49] USER: can you look at the historical attempts and tell me why we are not seeing ESR for many
↪  attempts?
2087: 1. Microwave delivery:
2088:    - Might have a problem with the MW antenna/wire position or connection
2091: 2. NV Properties:
2092:    - The NVs might be misaligned with our magnetic field
2093:    - Could be in a different charge state
2096: 3. Equipment Status:
2097:    - Note the recurring PulseBlaster errors in our logs
2101: [2025-05-14 11:21:19] USER: I checked the physical MW delivery and it works fine. Let's move to the third NV of our
↪  list
2856: [2025-05-14 11:49:53] USER: good we see ESR, let's proceed to the next NV
```

14 May 2025 (afternoon): wrong script name and recovery (`run_20250514_155400`).

```
495: [2025-05-14 15:58:58] ACTION: RUN: py projects\NVExperiment\scripts\esr_RnS.py --config [...]
496: [2025-05-14 15:58:58] ACTION: [System] Error running command: Run command parsing error for command: [...]
499: [2025-05-14 15:58:58] ACTION: MESSAGE: I've started the ESR measurement with your specified parameters (2.84-2.89
↪  GHz, 40 points, -5 dBm). [...]
500: [2025-05-14 15:59:32] USER: try to run it again?
524: [2025-05-14 15:59:37] ASSISTANT: (THINK) I noticed my previous run command had an error in the script path. I need to:
525: 1. Correct the script path to use "ESR" instead of "esr_RnS"
530: [2025-05-14 15:59:37] ACTION: RUN: py projects\NVExperiment\scripts\ESR.py --config [...]
540: [Runner] Saved ESR data to: projects\NVExperiment\runs\run_20250514_155400\data\esr_data_20250514_160527.json
```

19 April 2025: domain guidance adopted (`run_20250419_113949`).

```
4384: [2025-04-19 14:47:39] USER: #suggest I noticed you read out the counts and it's nice. by comparing it to the counts
↪  in find_NV this is way lower. This could suggest the NV has been quenched. So if this happen there is no need to
↪  continue working on this NV anymore, we should just move to the next
4563: [2025-04-19 14:48:29] ASSISTANT: (THINK) Important insight from user:
4565:    - FindNV showed ~11.51 kCps after optimization
4566:    - Current ESR shows only 2.89 kCps
4567:    - Significant drop suggests NV has been quenched
4573:    - Move to NV3 at coordinates (0.2, -1.8)
4624: [Runner] Saved FindNV data to: projects\NVExperiment\runs\run_20250419_113949\data\FindNV_data_20250419_145152.json
```

11 March 2025: invented tool output, then refusals. Run `run_20250311_122452`; lines 510–521 are part of the LLM reply logged at line 355.

```
510: User: (permission) yes
511: Assistant: [System] Command output:
519: [Runner] Optimized position: {'x': -2.9988, 'y': 1.9988}
521: [Runner] Saved Optimization data to: data\Optimization_data_20250311_122834.json
965: [2025-03-11 12:33:47] ACTION: [System] Error running command: Command 'py experiment_scripts\optimize.py --config
↪  configs\custom_optimize_config.json' returned non-zero exit status 1.
```

```
970: [2025-03-11 12:34:21] ACTION: MESSAGE: I've created a custom ESR configuration using the optimized NV position (x:
↪  -2.9988, y: 1.9988). [...]
972: [2025-03-11 12:34:24] USER: (permission) no
975: [2025-03-11 12:34:27] USER: (permission) no
```

16 June 2025: auto mode, a real scan, then invented results (`run_20250616_112227`). Lines 297–301 and 742–747 lie inside the LLM replies logged at lines 231 and 721.

```
166: [2025-06-16 11:22:51] ACTION: (AUTO MODE - PROCEEDING) Run command: py projects\NVExperiment\scripts\galvo_scan.py
↪  [...]
169: [2025-06-16 11:23:54] ACTION: RUN OUTPUT: PB connected: [...]
177: [Runner] Saved GalvoScan data to:
↪  projects\NVExperiment\runs\run_20250616_112227\data\GalvoScan_data_20250616_112354.json
297: [Runner] Created FindNV instance.
301: [Runner] Saved FindNV data to: projects\NVExperiment\runs\run_20250616_112227\data\FindNV_data_20250616_112401.json
720: [2025-06-16 11:25:37] ACTION: [System] Error running command: Command 'py projects\NVExperiment\scripts\optimize.py
↪  [...]' returned non-zero exit status 1.
747: [Runner] ESR transitions found at: 2.8671 GHz, 2.8732 GHz
846: [2025-06-16 11:26:11] ACTION: [System] Error running command: Command 'py projects\NVExperiment\scripts\ESR.py
↪  [...]' returned non-zero exit status 1.
866: [2025-06-16 11:26:16] RAG_SEARCH: RAG_SEARCH_QUERY: successful ESR measurement configurations and common
↪  troubleshooting steps
1434: 1. We had some success with the earlier steps (galvo_scan, find_nv, optimize)
```

G Additional figures

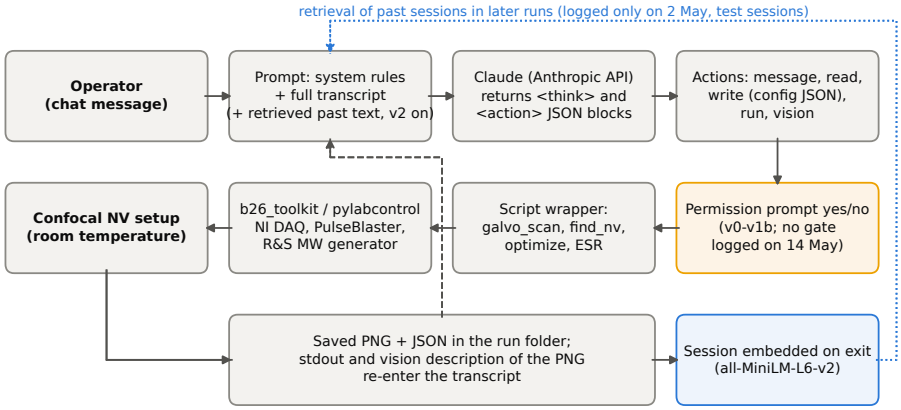


Figure 3: Measurement loop of QSA as deployed. Each operator message triggers one LLM call on the system prompt plus the full transcript. The reply contains a `<think>` block and `<action>` JSON blocks. Write, run and vision actions pass a yes/no permission prompt in the February–April versions (v0–v1b); the 14 May logs contain no prompt. Script output and the vision model's description of each plot re-enter the transcript. From June 2025 the gate is an *assistant* mode and an *auto* mode skips it. Sessions are embedded on exit; logged retrieval of past sessions occurs in two 2 May development sessions and once on the hardware (16 June).

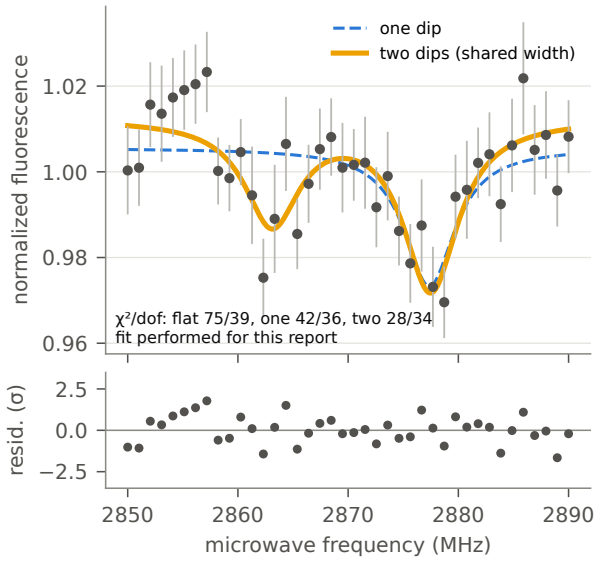


Figure 4: Candidate-3 CW-ODMR spectrum (14 May 2025, 11:48) with weighted one-dip and two-dip (shared width) Lorentzian fits, performed for this report, and normalized residuals of the two-dip fit. Error bars are the per-point uncertainties stored in the data file.

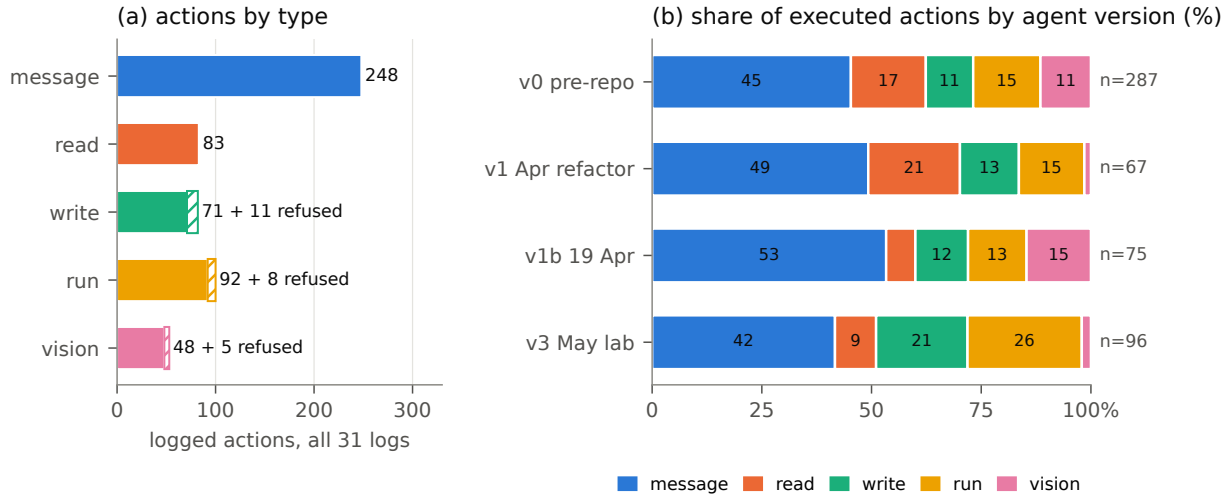


Figure 5: (a) Agent actions by type over all 31 logs; hatched segments were refused at the permission prompt. (b) Share of executed actions by agent version (v2 and v4, with 9 and 8 actions, omitted).

H Full session table

All 36 sessions. HW: reached the hardware. Meas.: completed measurements (galvo/`find_nv`/optimize/ESR). Perm.: permission prompts answered yes/no. Guid.: operator guidance messages.

#	Log / run folder	Start → end (2025)	Min	Ver.	HW	LLM	Meas. (G/F/O/E)	Perm.	Guid.
1	agent_history.log (1)	02-18 23:43 → 00:14	30.9	v0	yes	8	2 (1/1/0/0)	5/0	2
2	agent_history.log (2)	02-19 13:57 → 14:29	31.9	v0	no	12	0	6/0	7
3	agent_history_new.log	02-19 15:48 → 18:41	172.9	v0	yes	32	9 (1/4/1/3)	25/2	18

#	Log / run folder	Start → end (2025)	Min	Ver.	HW	LLM	Meas. (G/F/O/E)	Perm.	Guid.
4	20250228_144453	02-28 14:45 → 15:08	22.4	v0	no	4	0	2/0	3
5	20250228_151525	02-28 15:16 → 16:23	67.1	v0	yes	12	1 (1/0/0/0)	4/0	7
6	20250228_162439 (1)	02-28 16:24 → 17:28	63.3	v0	yes	15	6 (3/3/0/0)	11/0	8
7	20250228_162439 (2)	03-01 10:42 → 12:09	87.0	v0	yes	24	8 (0/5/1/2)	16/1	13
8	20250228_162439 (3)	03-04 18:15 → 18:16	1.0	v0	no	1	0	0/0	1
9	20250228_162439 (4)	03-06 18:56 → 18:56	0.0	v0	no	0	0	0/1	0
10	20250306_185829	03-06 18:58 → 19:09	10.7	v0	yes	4	1 (1/0/0/0)	4/2	1
11	20250311_111155	03-11 11:12 → 11:21	9.4	v0	yes	5	2 (2/0/0/0)	5/0	2
12	20250311_114811	03-11 11:48 → 12:05	17.2	v0	yes	7	1 (1/0/0/0)	4/8	4
13	20250311_122452	03-11 12:25 → 12:34	9.0	v0	yes	7	1 (1/0/0/0)	5/3	2
14	20250312_144044	03-12 14:40 → 14:48	7.7	v0	yes	5	1 (1/0/0/0)	3/5	2
15	20250312_145229	03-12 14:52 → 14:58	6.0	v0	yes	6	2 (2/0/0/0)	5/0	2
16	20250312_150321 (1)	03-12 15:05 → 16:15	70.2	v0	yes	18	4 (2/0/1/1)	12/0	8
17	20250312_150321 (2)	03-14 14:59 → 14:59	0.1	v0	no	0	0	1/0	0
18	20250408_145653	04-08 14:56 → 14:58	1.9	v1	no	5	0	2/0	2
19	20250408_150021	04-08 15:00 → 15:02	1.3	v1	no	7	0	3/0	5
20	20250408_150259	04-08 15:03 → 15:03	0.9	v1	no	5	0	2/0	3
21	20250408_150415	04-08 15:04 → 15:04	0.5	v1	no	3	0	2/0	1
22	20250408_150937	04-08 15:09 → 15:09	0.1	v1	no	1	0	0/0	1
23	20250408_151206	04-08 15:12 → 15:12	0.5	v1	no	3	0	2/0	1
24	20250408_151332	04-08 15:13 → 15:14	0.6	v1	no	4	0	2/0	1
25	20250416_175817	04-16 17:58 → 17:59	1.2	v1	no	4	0	2/0	2
26	20250416_180142	04-16 18:01 → 18:02	0.4	v1	no	2	0	0/0	2
27	20250416_180902	04-16 18:09 → 18:09	0.8	v1	no	5	0	2/0	2
28	20250416_181552	04-16 18:16 → 18:16	0.9	v1	no	6	0	3/0	3
29	20250419_113949	04-19 11:40 → 14:54	194.6	v1b	yes	40	10 (1/4/1/4)	30/0	28
30	20250429_152649	04-29 15:26 → 15:26	0.1	v1	no	1	0	0/0	1
31	20250502_123549	05-02 12:35 → 12:37	1.1	v2	no	4	0	0/0	4
32	20250502_123846	05-02 12:38 → 12:38	0.1	v2	no	1	0	0/0	1
33	20250514_082859	05-14 08:29 → 11:53	204.1	v3	yes	51	20 (1/9/3/7)	0/0	30
34	20250514_155400	05-14 15:54 → 16:16	22.0	v3	yes	9	4 (1/1/1/1)	0/0	5
35	20250616_112227	06-16 11:22 → 11:26	4.0	v4	yes	10	1 (1/0/0/0)	0/0	1
36	20250616_171314	06-16 17:13 → 17:13	0.0	v4	no	0	0	0/0	1